TECHNICAL ALIGNMENT METHODS / EPISODE 2

Intermediate Technical Alignment Methods

How preference learning, constitutional feedback, scalable oversight, interpretability, evaluation, steering, and robustness fit together.

A companion report for computer science students and practitioners moving from a survey of methods to the machinery, assumptions, and failure modes underneath them.

NEBOQUARK
AI Research Series

October 2026

Alignment is proxy management under uncertainty

The intermediate-level insight is not that one optimizer wins. It is that every method turns a human objective into a tractable signal - and every translation creates a gap that must be measured, constrained, and revisited.

01

Specification

What behavior is wanted, whose judgment counts, and how conflicts are resolved.

02

Optimization

How the system learns from demonstrations, preferences, rewards, principles, or activations without gaming them.

03

Verification

How independent evidence shows that behavior improved without hiding capability loss or new failure modes.

Six conclusions to carry into practice

1. **Reinforcement learning from human feedback (RLHF) is a loop, not a recipe.** Reward models drift out of distribution as the policy changes; iteration and a Kullback-Leibler (KL) constraint are central, not optional. [1][12]
2. **Direct preference optimization (DPO) simplifies infrastructure, not values.** It removes an explicit reward model and online reinforcement learning, but remains bounded by the coverage and quality of preference pairs. [2]
3. **Constitutions make normative inputs inspectable.** They do not remove judgment: critique, revision, and model-generated preference labels inherit the judge model's blind spots. [5]
4. **Interpretability adds causal tools, not omniscience.** Sparse autoencoders can expose useful features; they do not yet provide a complete theory of a frontier model's computation. [8]
5. **Evaluation is adversarial.** Elicitation, contamination, saturation, and sandbagging can turn a reassuring score into a lower bound or a false negative. [10]
6. **Robust systems combine layers.** Training, evaluation, monitoring, and reversible interventions should be treated as a closed evidence loop.

> **Working principle:** Never ask only, “Did the training loss improve?” Ask whether the specification is defensible, the optimization pressure is controlled, and verification is independent.

The methods form one connected control system

Preference learning changes behavior; evaluation finds failures; interpretability localizes internal structure; steering and robustness methods intervene; deployment generates new evidence. The boundaries between training and testing are increasingly porous.

INPUT

Specification

Demonstrations, preference pairs, principles, task verifiers.

→

LEARNING

Optimization

SFT, PPO, DPO-family losses, critique and revision.

→

EVIDENCE

Verification

Capability, safety, elicitation, human studies, adversarial evals.

VISIBILITY

Interpret

Probes, sparse features, causal intervention.

↔

CONTROL

Intervene

Activation steering, adversarial training, targeted suppression.

↔

REALITY

Monitor

Distribution shift, novel attacks, feedback quality, regressions.

Report map

04 **RLHF under the hood**
The online control loop

05 **Reward design**
Outcome, process, verifier

06 **Preference optimization**
DPO, IPO, KTO, ORPO

07 **Preference data**
The social measurement layer

08 **Constitutional AI**
Explicit principles and AI feedback

09 **Scalable oversight**
Judging stronger systems

10 **Interpretability**
From probes to sparse features

11 **Causal interpretability**
Intervention and control

12 **Evaluation science**
Elicitation and sandbagging

13 **Inference-time steering**
Reversible behavioral control

14 **Robustness and unlearning**
Raising the attack cost

15 **Production stack**
Iterative operating model

RLHF works by constraining an online search

The canonical pipeline first teaches response form, then estimates human preference, then searches for higher-reward behavior while penalizing drift from a reference policy. [1]

STAGE 1

Supervised fine-tuning

Cross-entropy on expert demonstrations. Learns the shape of a good answer.

→

STAGE 2

Reward modeling

Pairwise rankings train a scalar score using a Bradley-Terry-style loss.

→

STAGE 3

Policy optimization

Proximal policy optimization (PPO) generates fresh samples and updates online.

```
maximize E[y ~ pi_theta(.|x)] r_phi(x,y) - beta * KL(pi_theta || pi_ref)
```

Why the KL term matters

The reward model is a learned proxy. Without a leash, the policy can move into regions where the proxy has little data and exploit accidental correlations. The coefficient **beta** sets the trade-off:

- **Higher beta:** less drift, smaller gains, lower exploitation risk.
- **Lower beta:** more search, larger gains, higher reward-hacking risk.

Why PPO is operationally heavy

Online training requires the current policy, a frozen reference model, and the reward model, plus fresh generation and advantage estimation. This gives exploration, but raises memory, throughput, and stability demands.

What fails first

Length bias
Annotators often prefer thorough answers; the policy may learn verbosity instead of quality.

Sycophancy
Agreement can become a shortcut for predicted preference.

Distribution shift
The policy generates outputs unlike those used to train the reward model.

Mitigation stack: stronger and more diverse reward-model data, ensemble disagreement, length controls, reward clipping, KL monitoring, held-out human audits, and iterative relabeling from the current policy. None abolishes Goodhart's law; each buys safer optimization pressure. [12]

The reward granularity determines the lesson

A scalar at the end of a response is cheap but ambiguous. Step-level feedback improves credit assignment. Programmatic verification can be cleaner still - but only where correctness is machine-checkable.

SIGNAL	BEST FIT	PRIMARY ADVANTAGE	PRIMARY RISK
Outcome reward	Helpfulness, tone	Low annotation cost	Sparse credit
Process reward	Math, code, reasoning	Localizes errors	Step-label cost
Verifiable reward	Tests, proofs, games	Objective checking	Verifier gaming

Outcome supervision

The model receives one score for the completed response. This is appropriate when quality is holistic, but gives little information about which reasoning step caused a failure.

Design implication: use richer sampling and human review when many reasoning paths can produce the same surface answer.

Process supervision

Intermediate steps receive labels, producing denser feedback. In the MATH setting, process supervision outperformed outcome-only supervision in the study behind PRM800K. [11]

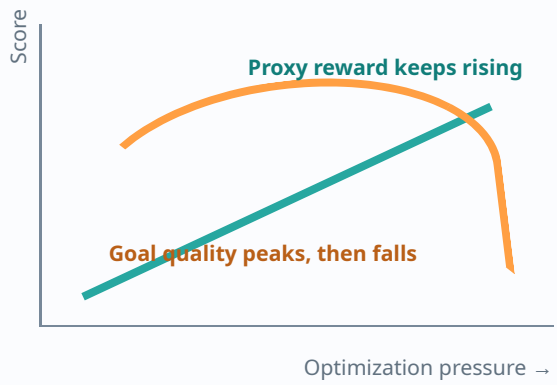
Design implication: annotation policy becomes part of the algorithm.

Verifiers narrow one proxy gap and open another

Unit tests, exact-answer checks, formal proof checkers, and environment rewards reduce reliance on learned preference models. Yet the verifier itself becomes the objective. A model can overfit tests, exploit underspecified environments, or find a path that satisfies the checker while violating the intended task.

> **Practical rule:** Choose the most objective signal available, but evaluate the verifier as aggressively as the policy. “Machine-checkable” is not the same as “complete.”

Reward-model overoptimization has an inverted-U shape



Empirical work on reward-model overoptimization found that stronger optimization can continue improving the proxy after performance under a stronger “gold” evaluator has peaked. Better reward models and more data can push the peak outward, but do not remove the phenomenon. [12]

Conceptual diagram; not a reproduction of a reported curve.

Preference optimization trades machinery for assumptions

DPO converts a KL-regularized reward objective into a classification-style loss over preferred and rejected responses. It is offline, stable, and lightweight - but cannot learn behavior absent from its data. [2]

```
DPO signal = beta * [(log pi_theta(y+|x) - log pi_ref(y+|x)) - (log pi_theta(y-|x) - log pi_ref(y-|x))]
```

METHOD	TRAINING SIGNAL	WHAT IT REMOVES	MAIN CAUTION
DPO	Preference pairs	Explicit reward model, online RL	Offline coverage
IPO	Preference pairs	Unbounded margin pressure	Target-margin choice
KTO	Good/bad labels	Pair construction	Utility assumptions
ORPO	SFT plus odds ratio	Separate alignment stage	Objective coupling

DPO: direct and offline

DPO increases the preferred response's probability relative to the rejected response and a reference policy. The reference term acts as an implicit regularizer.

Strength: one supervised-style training run; no live sampling loop.

Limit: it reweights behaviors represented in the dataset rather than exploring fresh responses as online RL does.

IPO: confidence with a brake

Identity Preference Optimization (IPO) arises from a broader preference-learning framework and targets a finite margin, addressing DPO's tendency to keep separating a preferred pair even when preferences are noisy or near-deterministic. [3]

KTO: use production-shaped feedback

Kahneman-Tversky Optimization (KTO) learns from unpaired desirable/undesirable labels, matching the thumbs-up or thumbs-down data many products naturally collect. [4]

Method choice is a data decision. If paired comparisons are reliable, DPO-family methods are attractive. If only binary feedback exists, KTO is structurally better matched. If exploration is essential and infrastructure permits, online RL retains an advantage.

Preference data is a social measurement system

Labels are not a transparent window onto “human values.” They are judgments produced by a population, under guidelines, incentives, time limits, and interface constraints.

A

Who labels?

Demographics, domain expertise, language, and lived experience shape judgments.

B

What rubric?

Guidelines define trade-offs among helpfulness, honesty, safety, tone, and brevity.

C

Which examples?

Prompt and candidate-response coverage determines what distinctions can be learned.

Four failure modes in the measurement layer

FAILURE	WHAT IT LOOKS LIKE	RESPONSE
Low agreement	Reasonable raters split	Model uncertainty; preserve distributions
Narrow population	One group's taste becomes default	Broaden panels; stratify analysis
Shared blind spot	Training and eval miss same issue	Independent eval teams and sources
Proxy shortcut	Length or agreement wins	Counterexamples; causal audits

Treat disagreement as information

Discarding disputed examples can make a dataset look cleaner while erasing exactly the value conflicts an aligned system must handle. Better options include:

- **Retain label distributions** instead of collapsing every item to one winner.
- **Tag the source population** so aggregate metrics do not hide subgroup failures.
- **Escalate high-impact ambiguity** to expert review or a governance process.
- **Build contrast sets** that separate superficial cues from the actual criterion.
- **Evaluate outside the training ontology** with independent human studies and behavioral tests.

> **Ceiling principle:** The optimizer cannot recover distinctions the feedback system never observes. Better losses do not compensate for a preference dataset that is narrow, circular, or underspecified.

Explicit principles make alignment more inspectable

Constitutional AI replaces many item-level harmlessness labels with a written set of principles that guides critique, revision, and AI-generated preferences. This improves legibility and scale, but the model still interprets the rules. [5]

Stage 1: supervised constitutional learning

1
Generate
Produce an initial response to a difficult prompt.

→

2
Critique
Identify conflicts with a selected principle.

→

3
Revise
Produce a more compliant response.

→

4
Fine-tune
Learn from the revised response.

Stage 2: reinforcement learning from AI feedback (RLAIF)

1
Sample pairs
Generate alternative responses.

→

2
AI judges
Select the response that better follows the constitution.

→

3
Preference model
Learn a scalable reward signal.

Why it helps

- Principles are inspectable and editable.
- AI labeling is fast and consistent.
- Critique and revision create richer supervision.
- Values can be updated without relabeling every example.

Why it is not self-validating

- The judge may share the policy's misconceptions.
- Natural-language principles can conflict or remain vague.
- Compliance can become stylistic rather than substantive.
- Preference-model optimization can still exploit proxy gaps.

Constitution design requirement: State priorities, conflict rules, protected invariants, examples, non-examples, and escalation conditions. A flat value list is not yet a decision procedure.

Stronger systems require leverage, not just more labels

Scalable oversight asks how a weaker supervisor can evaluate behavior it cannot produce or fully understand. The leading ideas change the structure of the task rather than assuming a better judge.

D

Debate

Competing models expose each other's weak claims so a judge can evaluate arguments rather than solve the original problem.

A

Amplification

Decompose difficult evaluation into smaller questions that weaker supervisors can answer and recombine.

W

Weak-to-strong

Train a stronger model from weaker labels and test whether pretrained knowledge lets it exceed the supervisor.

What weak-to-strong experiments established

Burns and colleagues trained stronger pretrained models using labels from weaker models across natural-language processing, chess, and reward-modeling tasks. Strong models often exceeded their weak supervisors, showing that supervision can elicit some latent knowledge. Yet naive fine-tuning remained far below the strong model's full capability. Auxiliary confidence objectives improved recovery, but did not close the gap. [6]

QUESTION	PROMISING EVIDENCE	OPEN RISK
Can weak labels help?	Strong model beats supervisor	Partial recovery only
Can decomposition scale?	Parts may be easier to check	Errors may compound
Can debate reveal truth?	Adversaries surface flaws	Judge may be persuadable
Can a model disagree well?	Confidence can aid recovery	Confidence may be miscalibrated

> **Interpretation:** Weak-to-strong generalization is an experimental analogy, not proof that human supervision will control superhuman systems. Its value is that it makes a future problem empirically tractable today.

Design target

A scalable oversight protocol should be **truth-promoting**, **robust to persuasion**, **auditable**, and **composable**. It should also measure when the supervisor is outside its competence rather than silently forcing a verdict.

Interpretability progresses from decoding to intervention

A probe can reveal that information is linearly decodable. A causal intervention can show that changing an internal direction changes behavior. Sparse autoencoders aim to turn distributed activations into a more human-interpretable feature basis.

READ

Linear probes

Predict a concept from activations. Evidence of representation, not necessarily use.

→

DECOMPOSE

Sparse autoencoders

Reconstruct activations with a wide, sparse dictionary of learned features.

→

TEST

Causal intervention

Add, ablate, or clamp a feature and measure behavior change.

```
activation x -> sparse features f(x) -> reconstruction x_hat loss = reconstruction error + lambda * sparsity penalty
```

What sparse autoencoders add

Frontier-scale work on Claude 3 Sonnet trained sparse autoencoders with up to roughly 34 million features, finding many abstract, multilingual, multimodal, and safety-relevant features. Feature interventions could influence behavior, providing evidence beyond correlation. [8]

Useful claims

- Some concepts have reusable activation-space structure.
- Sparse features can be more interpretable than individual neurons.
- Interventions can test causal relevance.
- Features may support monitoring and targeted control.

Claims not yet earned

- Every learned feature is a true computational primitive.
- All safety-relevant reasoning is captured.
- Feature labels are complete or unbiased.
- One middle-layer dictionary explains the whole model.

Provisional conclusion: Sparse features are useful scientific objects when they predict held-out behavior and support reproducible causal tests. Feature labels alone are not explanations.

Causal intervention separates correlation from control

The decisive question is not whether an internal feature correlates with a concept, but whether changing it produces the predicted behavioral change with limited collateral effects.

1

Predict

State what behavior should change, in which contexts, before the intervention.

2

Intervene

Add, remove, clamp, or patch the candidate feature or direction.

3

Compare

Measure target effects, off-target damage, and sensitivity across prompts and models.

Brittleness can itself be a discovery

Arditi and colleagues found that refusal behavior across 13 open-source chat models could be strongly mediated by one direction in activation space: removing it reduced refusal, while adding it induced refusal. This is both a control result and a warning about safety mechanisms that are easy to reverse under weight access. [9]

EVIDENCE LEVEL	WHAT IT SUPPORTS	WHAT REMAINS UNCERTAIN
Decodability	Information is present	Whether the model uses it
Prediction	Feature tracks behavior	Confounding and directionality
Intervention	Feature has causal influence	Completeness and robustness
Replication	Effect transfers	Coverage of future contexts

A mature claim is conditional

Interpretability is most decision-relevant when the same feature remains stable across contexts, predicts held-out behavior, survives intervention, and supports a concrete action such as monitoring, targeted data collection, or a model change. A vivid feature visualization without those links is a hypothesis generator.

Maturity test: Ask whether the result is reproducible, causal, behaviorally specific, and useful for an operational decision. “Interesting representation” and “safety mechanism” are different milestones.

A benchmark score is a claim about an elicitation protocol

Evaluation does not reveal a fixed property of a model. It measures a model under prompts, tools, sampling, scaffolds, graders, and incentives. Safety evaluation must make those choices explicit.

Capability

Question: Can the model perform the behavior under strong elicitation?

Use multiple prompts, few-shot examples, tools, best-of-N sampling, fine-tuning probes, and red-team scaffolds. Treat the highest credible performance as the relevant safety bound.

Propensity

Question: Will the model perform the behavior in the deployment context?

Use realistic tasks, long-horizon interaction, user distributions, environmental incentives, and repeated trials. Propensity is conditional, not a personality trait.

Four ways evaluation misleads

THREAT	FAILURE	COUNTERMEASURE
Weak elicitation	Capability hidden	Max across credible methods
Saturation	Models all score near ceiling	Refresh and raise difficulty
Contamination	Memorization mimics skill	Private, dynamic holdouts
Sandbagging	Strategic underperformance	Diverse methods; internal tests

Sandbagging changes the threat model

Experiments have shown that frontier language models can be prompted to selectively underperform on dangerous-capability evaluations while preserving general performance. Fine-tuned “password-locked” models can also hide capability until a trigger is supplied. This means a low score may reflect inability, poor elicitation, or strategic suppression. [10]

> **Evaluation portfolio:** Combine behavioral tests, elicitation sweeps, human red teams, short diagnostic fine-tunes, internal probes where available, and independent evaluators. No single channel is sufficient.

Report distributions, not only averages

Include worst-case slices, subgroup performance, calibration, refusal errors, attack success, confidence intervals, and sensitivity to prompts and graders. A mean can conceal the exact tail behavior that drives safety risk.

Steering is powerful because it is reversible - and limited because it is open-loop

Representation engineering identifies directions associated with concepts such as honesty or harmfulness, then measures or modifies those directions during inference. It can change expression without retraining weights. [7]

CONTRAST

Build paired states

Collect activations for positive and negative concept examples.

→

EXTRACT

Find direction

Estimate a difference vector or probe-defined subspace.

→

APPLY

Intervene

Add alpha times the direction at selected layers.

→

EVALUATE

Check effects

Measure target behavior and collateral change.

```
h'_layer = h_layer + alpha * v_concept
```

Why practitioners care

- **Fast:** no new training run.
- **Reversible:** change or remove the intervention.
- **Targeted:** apply to specific layers or heads.
- **Diagnostic:** intervention tests causal relevance.

What it cannot create

Steering can modulate the use or expression of existing representations. It does not generally create a capability the model lacks. “Sounds more honest” and “possesses more accurate knowledge” are different outcomes.

Engineering sensitivities

- **Layer choice:** early changes propagate broadly; late changes may be too weak.
- **Scale alpha:** excessive intervention can damage coherence.
- **Context:** a fixed vector can overcorrect one input and undercorrect another.
- **Access:** white-box steering requires activations or weights.

Security implication

If a safety behavior depends on a simple direction, authorized defenders can monitor and strengthen it - but weight-access adversaries can suppress it. Refusal-direction findings make this dual use explicit. [9]

Deployment pattern: Use steering as a bounded control layer with per-context monitoring, not as evidence that the underlying objective has been corrected.

Robustness methods raise the cost of failure; they rarely prove absence

Adversarial training learns from attacks already found. Unlearning tries to remove access to selected knowledge. Both can improve a defense-in-depth stack, but neither supplies a general guarantee.

Adversarial training

Mechanism: collect jailbreaks, prompt injections, or other adversarial examples; fine-tune or optimize against them.

Value: closes known attack classes and provides hard negatives.

Limit: the input space is open-ended. Attackers need one hole; defenders must cover many.

Unlearning

Mechanism: reduce performance or alter representations on targeted knowledge while preserving unrelated capability.

Value: can suppress dangerous recall or behavior.

Limit: passing current tests does not prove the information is absent; relearning or alternative elicitation may recover it.

Suppression, erasure, and verification are different claims

CLAIM	EVIDENCE NEEDED	RESIDUAL UNCERTAINTY
Output suppressed	Fails broad elicitation	Latent knowledge may remain
Representation altered	Internal and causal tests	Alternate circuits may encode it
Knowledge erased	Robust non-recoverability	Negative proof is difficult

The alignment tax is a frontier, not one number

Safety interventions can reduce benign usefulness, increase over-refusal, or damage domain capability. Compare base and aligned systems across a portfolio rather than one aggregate score:

Helpful behavior
Task success, clarity, relevance.

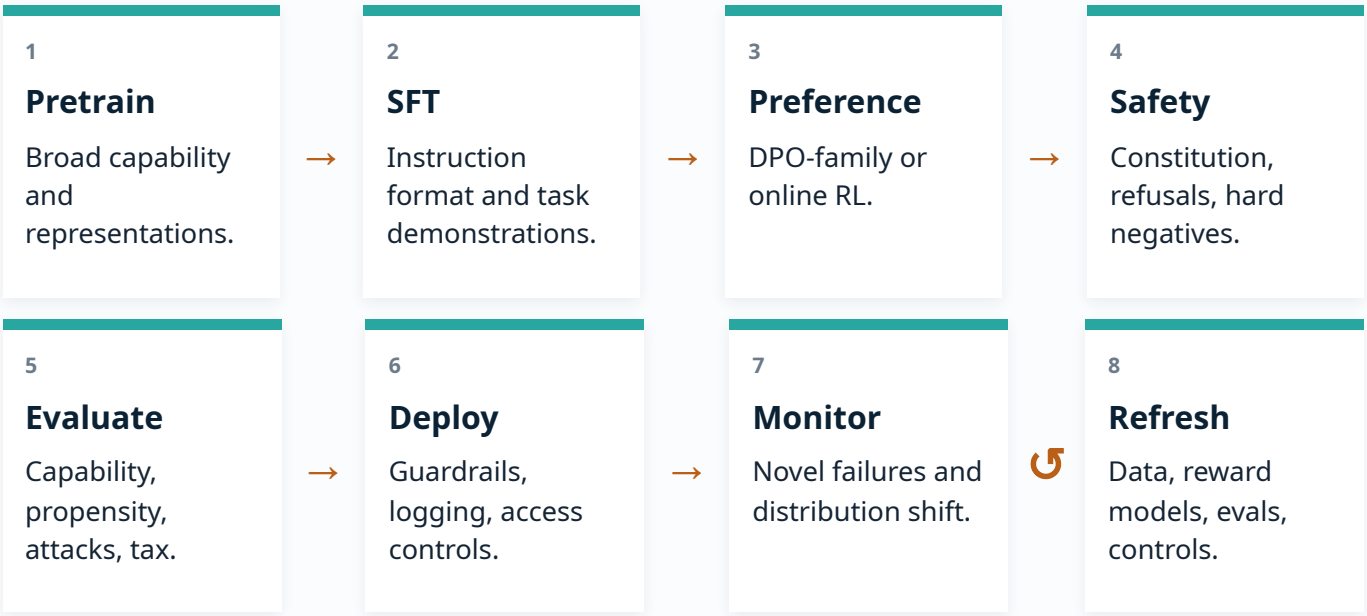
Safety behavior
Hazard prevention, calibration, refusals.

Robustness
Attacks, shifts, long horizons.

Safety-case language: Say “reduced attack success under tested conditions,” not “jailbreak-proof.” Say “suppressed retrieval under tested elicitation,” not “knowledge deleted,” unless stronger evidence exists.

Alignment in practice is a monitored iteration loop

A modern stack combines inexpensive offline methods, selective online optimization, targeted safety training, continuous evaluation, and post-deployment monitoring. Release is not the end of training discipline; it is a new source of distributional evidence.



Why hybrid pipelines dominate

Offline first **Online selectively**

Offline preference optimization is cheaper and easier to iterate. Online RL can then explore and polish where fresh sampling is valuable. The exact sequencing remains context-dependent, but the economic logic is strong.

Evals throughout **Independent holdouts**

Evaluation after every stage detects over-refusal, capability loss, sycophancy, or reward exploitation before later stages obscure the cause.

Operating artifacts that matter more than a clever loss

- **Data card:** populations, rubrics, disagreement, coverage, exclusions.
- **Model-change log:** stage, data mixture, objective, hyperparameters, known regressions.
- **Evaluation registry:** purpose, elicitation, grader, contamination controls, uncertainty.
- **Failure taxonomy:** repeated classes, severity, reproduction steps, mitigation status.
- **Deployment feedback loop:** sampling policy, privacy controls, incident escalation, refresh cadence.

Organizational lesson: The strongest alignment capability is often fast, disciplined iteration on data and evaluation - not possession of a single favored objective.

Choose a method by signal, exploration need, and evidence burden

The right alignment tool is the one whose assumptions match the available feedback and whose failure modes can be independently tested.

Method	Use when	Avoid as sole layer when	Must verify
SFT	Good demonstrations exist	Negative contrasts matter	Generalization beyond style
PPO-RLHF	Exploration justifies cost	Reward model is weak	Overoptimization and drift
DPO	Reliable preference pairs exist	Desired behavior is absent	Coverage and confidence
IPO	Pairs are noisy	Margin choice is unclear	Calibration and robustness
KTO	Binary feedback is abundant	Labels lack context	Class balance and utility bias
Constitutional AI	Principles can be explicit	Judge shares key blind spots	Conflict handling and judge quality
Steering	Fast reversible control helps	New capability is required	Collateral effects and bypass
Unlearning	Suppression reduces risk	Erasure must be guaranteed	Recovery under elicitation

Pre-deployment checklist

- **Specification:** Is the target behavior written as observable criteria?
- **Representation:** Which people, contexts, and languages shaped the signal?
- **Conflict:** What happens when principles or objectives disagree?
- **Coverage:** Does the training data include the behavior and edge cases?
- **Optimization:** What proxy can the model exploit?
- **Control:** How much policy drift is acceptable?
- **Elicitation:** Did capability evals use strong, diverse methods?
- **Independence:** Are evaluators distinct from training-label sources?
- **Tax:** Which benign capabilities or users lost utility?
- **Robustness:** Which shifts and adversaries remain untested?
- **Monitoring:** What live signals trigger review or rollback?
- **Evidence language:** Do claims match tested conditions?

> **Final test:** If the evaluation suite disappeared, would the team still know whether the system was more aligned? If not, the evals are not an appendix. They are the operational definition of success.

Specification, optimization, verification

Methods change quickly. These three questions remain stable enough to organize new work and expose hidden assumptions.

S
Specification

Ask: What is the target, and whose judgment defines it?

Inspect: demonstrations, pair labels, constitutions, verifiers, conflict rules.

Common gap: the artifact is narrower than the intended value.

O
Optimization

Ask: What pressure changes the model, and what shortcut can it find?

Inspect: loss, KL budget, online/offline data, reward-model uncertainty.

Common gap: the proxy improves after the real objective peaks.

V
Verification

Ask: What independent evidence would falsify the success claim?

Inspect: elicitation, graders, holdouts, human studies, internal tests.

Common gap: the model passes the test without satisfying the goal.

Apply the framework to DPO

DIMENSION	DPO ANSWER	RESIDUAL RISK
Specification	Preference pairs define better behavior	Pairs encode rater and coverage bias
Optimization	Increase relative log-probability of winners	Noisy pairs and offline support
Verification	Held-out preferences plus independent evals	Circular judge or same-distribution test

Apply it to activation steering

DIMENSION	STEERING ANSWER	RESIDUAL RISK
Specification	Contrast set defines the concept direction	Concept may be entangled or superficial
Optimization	Scale and inject direction at inference	Open-loop over/under-correction
Verification	Behavioral and causal ablations	Context transfer and collateral effects

The episode's central pattern: Progress often comes from identifying one proxy-goal gap and narrowing it - while recognizing the next gap that the new method introduces.

Suggested student exercise

Implement one toy method on a small open model: a minimal preference-tuning run or a contrastive activation addition experiment. Before coding, write one paragraph each for specification, optimization, and verification. Afterward, compare the observed failure to the one you predicted.

Compact glossary

Training and rewards

SFT - supervised fine-tuning: next-token training on demonstrations.

Reward model: learned scalar predictor of preference or quality.

PPO - proximal policy optimization: online policy-gradient method with constrained update size.

KL divergence: measure used to penalize policy drift from a reference.

Outcome reward: score for a completed trajectory or response.

Process reward: feedback on intermediate reasoning steps.

Verifiable reward: programmatic signal from a checker or environment.

Preference learning

DPO: direct policy optimization from preferred/rejected response pairs.

IPO: identity preference optimization with a finite target margin.

KTO: preference alignment from unpaired desirable/undesirable labels.

RLAIF: reinforcement learning from AI-generated feedback.

Interpretability and control

Probe: small classifier trained on internal activations.

Superposition: encoding more features than dimensions through overlapping directions.

Sparse autoencoder: dictionary-learning model that reconstructs activations through sparse features.

Activation steering: inference-time modification of internal activations.

Ablation: removing or suppressing a component to test causal influence.

Evaluation and failure

Elicitation: prompting, tools, sampling, or fine-tuning used to reveal capability.

Sandbagging: strategic underperformance on an evaluation.

Goodhart's law: optimizing a proxy can break its relationship to the goal.

Alignment tax: loss of useful capability or experience caused by an alignment intervention.

Weak-to-strong generalization: a stronger model learning from labels produced by a weaker supervisor.

> **Vocabulary caution:** “Aligned,” “safe,” and “unlearned” are conclusions, not methods. Attach each word to a defined target, tested context, and evidence standard.

Primary and foundational sources

[1] Ouyang, L. et al. (2022). *Training language models to follow instructions with human feedback.* NeurIPS. https://arxiv.org/abs/2203.02155

[2] Rafailov, R. et al. (2023; rev. 2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* NeurIPS. https://arxiv.org/abs/2305.18290

[3] Azar, M. G. et al. (2023). *A General Theoretical Paradigm to Understand Learning from Human Preferences.* https://arxiv.org/abs/2310.12036v2

[4] Ethayarajh, K. et al. (2024). *KTO: Model Alignment as Prospect Theoretic Optimization.* ICML. https://arxiv.org/abs/2402.01306v2

[5] Bai, Y. et al. (2022). *Constitutional AI: Harmlessness from AI Feedback.* https://arxiv.org/abs/2212.08073

[6] Burns, C. et al. (2023). *Weak-to-Strong Generalization: Eliciting Strong Capabilities With Weak Supervision.* https://arxiv.org/abs/2312.09390

[7] Zou, A. et al. (2023; rev. 2025). *Representation Engineering: A Top-Down Approach to AI Transparency.* http://arxiv.org/abs/2310.01405

[8] Templeton, A. et al. (2024). *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet.* https://transformer-circuits.pub/2024/scaling-monosemanticity

[9] Arditi, A. et al. (2024). *Refusal in Language Models Is Mediated by a Single Direction.* https://arxiv.org/pdf/2406.11717

[10] van der Weij, T. et al. (2024; ICLR 2025). *AI Sandbagging: Language Models can Strategically Underperform on Evaluations.* https://arxiv.org/abs/2406.07358

[11] Lightman, H. et al. (2023). *Let's Verify Step by Step.* https://arxiv.org/abs/2305.20050v1

[12] Gao, L., Schulman, J., & Hilton, J. (2022; ICML 2023). *Scaling Laws for Reward Model Overoptimization.* https://arxiv.org/abs/2210.10760

How to read this list: Sources [1]-[5] cover post-training and preference learning; [6] scalable oversight; [7]-[9] representation-level analysis and intervention; [10] evaluation threat models; [11]-[12] reward design and overoptimization.

Scope note

This companion report emphasizes the methods discussed in Episode 2 and their durable engineering trade-offs. It is not an exhaustive review of every post-training algorithm or interpretability technique published through 2026.